

Task 1

1a):

The maximum possible value M is the maximum value of $\text{bloomDay}[i]$, which is 10^9 .

Proof by contradiction: If $M > 10^9$, then there must be some flower that blooms after the 10^9 th day, or it would be possible to reduce the value of M

Simple test case: $(\text{bloomDay} = [10^9], m = 1, k = 1)$

To make a singular bouquet of size 1, we have to wait for the 10^9 th day for flower 0 to bloom.

1b)

Observation: If $\text{can}(d) \equiv T$ for some d , then $\forall h \geq d, \text{can}(h) \equiv T$ since the number of bloomed flowers never reduces.

Thus binary search may be used to find the minimum value of d where $\text{can}(d) \equiv T$.

```
let l be 1
let r be M
while r > l
    let mid be floor((l + r) / 2)
    if can(mid)
        let r be mid
    else
        let l be mid + 1
let d be l

if d == M and !can(d)
    return -1
else
    return d
```

Explanation:

If there is no minimum d in the range $[1, M]$ where $\text{can}(d) \equiv T$, then it is impossible to make m bouquets. Otherwise, the minimum d found must be the answer.

Time complexity analysis:

The initial search space is $[1, M]$. For each loop, $\text{can}()$ is called and the search space is halved. Thus each step takes $O(n)$ time and there can be up to $\log(M)$ steps. The total complexity for this algorithm would be $O(n \cdot \log(M))$.

1c)

Observation: Greedily picking flowers from the left in any contiguous segment of bloomed flowers must be optimal. This is because any range of unmatched bloomed flowers larger than k will lie to the right of the greedily selected bouquets.

Therefore a greedy algorithm from the left may be designed:

```
function can(d)
    let ctr be 0
    let currentSize be 0
```

```

for each i from 0 to n-1
    if bloomDay[i] <= d
        let currentSize be currentSize + 1
        if currentSize >= k
            let ctr be ctr + 1
            let currentSize be 0
    else
        let currentSize be 0

if ctr >= m
    return True
else
    return False

```

2a)

The maximum possible value M is achieved when a mechanic of the slowest working speed is assigned the maximum number of cars to repair.

Simple test case: ($rank_s = [100], cars = 10^6$)

In this case, $M = 100 * (10^6)^2 = 10^{14}$

2b)

let $can(d)$ be an $O(n)$ subroutine that verifies if it is possible for the mechanics to produce a total of $cars$ cars within d minutes.

Observation: Observation: If $can(d) \equiv T$ for some d , then $\forall k \geq d, can(k) \equiv T$, since the number of cars each mechanic will be able to repair can never reduce.

Thus binary search may be used to find the minimum value of d where $can(d) \equiv T$.

```

let l be 1
let r be M
while r > l
    let mid be floor((l + r) / 2)
    if can(mid)
        let r be mid
    else
        let l be mid + 1
let d be l
return d

```

Explanation:

Unlike **1b**), there is no case where the mechanics fail to repair all the cars within M minutes. Therefore the minimum value of d is simply l .

Time complexity analysis:

The initial search space is $[0, M]$. For each loop, $can()$ is called and the search space is halved. Thus each step takes $O(n)$ time and there can be up to $\log(M)$ steps. The total complexity for this algorithm would be $O(n \cdot \log(M))$.

2c) The time taken for the i^{th} mechanic to repair n cars t_i is given by:

$$t_i = ranks_i \cdot n^2$$

Therefore given a time limit of T minutes, the maximum number of cars that the i^{th} mechanic may repair is $\lfloor \sqrt{\frac{T}{ranks_i}} \rfloor$

Thus can(d) would be a simple sum: $\sum_{i=0}^{n-1} \lfloor \sqrt{\frac{d}{ranks_i}} \rfloor$

```
function can(d)
    let sum be 0
    for each i from 0 to n-1
        let a be floor(sqrt(d / ranks[i]))
        let sum be sum + a

    if sum < cars
        return false
    else
        return true
```

Task 2

1a)

Let the value stored by the i^{th} vertex (0-indexed) be v_i .

If there is only 1 vertex, the function will return v_0 with a probability of 0.5 (ie. the chance that $v \leq 0.5$). There is a probability of 0.5 that the function returns `None`.

If there are 2 vertices, the function will return v_0 with a probability of 0.25, v_1 with a probability of 0.5, and has a probability of 0.25 of returning `None`.

If there are n vertices (0-indexed), then:

$$P(\text{getRandom}() = v_i) = 2^{i-n}$$

This is because the i^{th} vertex can only be returned if all other vertices after it are not chosen. Additionally, the probability that `None` is returned is 2^{-n} .

This will not be accepted because:

1. `None` is not an accepted result for the function
2. The probability that each vertex is selected is not uniform

1b)

Single-line fix: Replace line 8 with the following statement:

```
if v <= 1 / ((i - 777) / 2)
```

Explanation: with this expression, the i^{th} vertex will be assigned to `ans` with a probability of $\frac{1}{(i+1)}$ (eg. v_0 will be assigned with a probability of 1, v_1 will be assigned with a probability of $1/2$, etc.).

This means that the final chance that any v_i is returned is

$$P(\text{getRandom}() = v_i) = \frac{1}{i+1} \cdot \frac{i+1}{i+2} \cdot \frac{i+2}{i+3} \cdot \dots \cdot \frac{n-2}{n-1} \cdot \frac{n-1}{n} = \frac{1}{n}$$

1. None will never be returned because the first vertex is assigned with a probability of 1.
2. The distribution is now uniform as every value may be returned with a probability of n^{-1}

2a)

Answer: $\frac{2^{n-1}}{n!}$

We assert that a permutation of an array of size n with distinct sorted elements $a_1, a_2, \dots, a_n$ can only be sorted by bubblesort in 2 passes iff it satisfies one of the following rules:

1. The array is a single element
2. The permutation follows the following pattern: $[a_n, a_1, a_2, \dots, a_{n-1}]$
3. The permutation can be split into smaller subarrays which satisfy Rule 1 or Rule 2.
 - For single-element subarrays, they must already be in the correct position for the final sorted array.
 - For subarrays satisfying Rule 2, all elements except the largest element must be 1 position to the right of their final positions.

Examples:

$[3, 1, 2, 7, 4, 5, 6, 8, 9]$ may be broken into the subarrays $[3, 1, 2]$, $[7, 4, 5, 6]$, $[8]$, $[9]$, where $[8]$ and $[9]$ are already in the correct positions for the final sorted array.

$[8, 3, 1, 2, 7, 4, 5, 6, 9]$ does not satisfy the requirements as $[8]$ violates Rule 3.1

$[6, 4, 5, 3, 1, 2, 7, 8, 9]$ does not satisfy the requirements as $[6, 4, 5]$ and $[3, 1, 2]$ violate Rule 3.2

Proof By Induction:

Firstly, observe the following for any permutation that may be sorted in 2 passes:

1. If any element is already in the correct position, no swap operations can be performed on it
 - Otherwise, the same swap operation needs to be performed in an additional pass to revert it
2. All elements can only be shifted to the left once during the first pass
 - Thus any element more than 1 position to the right of its final sorted position cannot exist

Base step:

An array of size 1, which is already sorted and satisfies Rule 1.

Inductive step:

Given an array a of size n that satisfies the conditions, we attempt to add the $n + 1^{th}$ element to the array:

1. If we append it to the right of the array, then Rule 1 is satisfied and the array may still be sorted in 2 passes.
2. If we insert to the middle of the array, the elements we shift to the right belong to subarrays satisfying Rule 1 or Rule 2.
 - If they currently satisfy Rule 2, then the insertion will cause elements to be 2 positions to the right of their final sorted position due to Rule 3.2. Therefore the permutation can no longer be sorted in 2 passes, and Rule 3.2 is similarly violated in the newly created array.
 - If all elements shifted to the right currently satisfy Rule 1, then we have created a new subarray satisfying Rule 2. The resulting permutation satisfies the conditions and can still be sorted in 2 passes.
3. Inserting to the left of the array results in the same observations as inserting to the middle of the array.

Therefore we prove by induction that these conditions are satisfied by any permutation of distinct integers that can be sorted in 2 passes.

We propose the following construction:

1. Partition a sorted array of distinct integers into subarrays $k_1, k_2, k_3, \dots$
2. For the i th subarray, if $|k_i| = 1$ then we do not modify it (Rule 1)
3. Else, we shift the largest element to the front of the array (Rule 2)

Proof of Bijectivity:

Surjectivity is already proven by our earlier induction (since every array of distinct integers that may be sorted in 2 passes is in the set of permutations assembled with partitions), thus we only need to prove injectivity for this construction. We can prove this by contradiction:

- Assume that 2 different subarray sequences exist on a sorted array that produce the same permutation
- Consider the first partition which differs in size between the two sequences
- Since one of the partitions is larger, then its first element in the resulting permutation must be different since the largest element is shifted to the front.

Thus the construction is bijective wrt. all possible permutations of an array that can be sorted in 2 passes.

To count how many possible permutations exist, we can simply count how many ways an array may be separated into subarrays. There are $n - 1$ possible partitions between n elements, thus 2^{n-1} such unique arrays.

Therefore the probability of a randomly generated array being sorted in 2 passes is $\frac{2^{n-1}}{n!}$.

3a)

Total area that (px, py) may lie in: $4r^2$

Total area within the circle: πr^2

Probability that (px, py) lies in the circle: $\frac{\pi}{4}$

$$\therefore P(\text{randPoint}() \neq \text{None}) = 1 - (1 - \frac{\pi}{4})^Z$$

3b)

At most $3 \cdot 10^4$ calls will be made to `randPoint()`.

If we want to be more likely to succeed than fail, we need the expectation of the number of failures after $3 \cdot 10^4$ calls to `randPoint()` to be less than 1:

$$(1 - \frac{\pi}{4})^Z \cdot 30000 < 1$$

$$Z > 6.6986$$

Therefore the smallest value of Z where we can expect to succeed is 7. However, this only guarantees that we are more *likely* to succeed:

$$P(\text{Succeed}) = (1 - (1 - \frac{\pi}{4})^Z)^{30000}$$

$$Z = 7, P(\text{Succeed}) \approx 0.53$$

Submitting this code still has a significant chance of failure. Ideally, Z should be infinity for the code to definitely succeed, but setting $Z \geq 12$ already gives us a 0.02% chance of failure.